

Báo cáo tổng hợp — Đánh giá các phương pháp sinh lỗi giả

Bài toán A1 — Predictive-Maintenance AI Despite Scarce Fault Data (DENSO Factory Hacks 2026)
Dataset chính: NASA IMS (run-to-failure) · **Nền tảng:** Random Forest + 24 feature kỹ thuật (z-score)
Harness đánh giá: cửa sổ 512 / stride 256 (~42ms), bóp số lỗi thật trong train (K), *test cố định 8100 lỗi thật*, baseline tái lập theo seed `seed+K` . So sánh **CÔNG BẰNG**: chỉ phần "lỗi giả" khác nhau.

1. Kết quả tổng quan

Recall trên test (mean 3 seed · interp_align+amp · k_mix=6 · n_synth=3200):

Số lỗi thật trong train (K)	Baseline (không augment)		Đóng góp
10	0.252	0.766	+0.516
20	0.383	0.748	+0.364
50	0.490	0.788	+0.266

Cách tính "Đóng góp": trung bình của (recall_sau – recall_trước) trên từng seed (3 seed) — cách đo sạch nhất, vì mỗi seed dùng cùng một test (before và after chấm trên đúng test đó). Nếu so $mean_after - mean_before$ thì cũng gần đúng (+0.516/+0.364/+0.266 so với mức mean 3 seed 0.173/0.375/0.512).

Câu chuyện cốt lõi: khi dữ liệu lỗi thật RẤT hiếm (10–50 mẫu), một bộ sinh lỗi giả tốt giúp recall tăng mạnh (gấp 2–3 lần, +0.27 → +0.52). Ngược lại khi đã có ≥100–200 lỗi thật, baseline chạm trần (IMS ~0.886) và augment gần như vô ích ($\Delta \approx 0$).

2. Bảng phân hạng từng phương pháp (recall sau aug, mean 3 seed)

Đây là số đã được kiểm chứng nhất quán. Baseline: K=10 = **0.252**, K=20 = **0.383**, K=50 = **0.490**.

#	Phương pháp	K=10	K=20	K=50	Mô tả ngắn
1	interp_align + amp + k_mix=6	0.766 ±0.021	0.748 ±0.012	0.788 ±0.025	interpolate + căn pha + chuẩn biên độ + trộn 6 cửa sổ
2	amp_align	0.678 ±0.058	0.733 ±0.010	0.773 ±0.031	interpolate + chuẩn biên độ
3	interpolate (optimized)	0.466 ±0.009	0.671 ±0.061	0.724 ±0.038	trộn 2 lỗi thật + jitter

#	Phương pháp	K=10	K=20	K=50	Mô tả ngắn
4	heuristic ($\times 2.5$ + nhiễu)	0.492 ± 0.031	0.478 ± 0.029	0.628 ± 0.018	phóng đại biên độ normal
5	physics (BPFO/BPFI/BSF/FTF)	0.411 ± 0.031	0.440 ± 0.007	0.597 ± 0.009	mô phỏng cơ chế hỏng ổ bi
6	DDPM / Diffusion	0.432 ± 0.035	0.435 ± 0.009	0.604 ± 0.009	generative (SOTA)
7	TimeGAN (generative)	—	~ 0.43	~ 0.58	GAN theo thời gian
8	aligned (ECDF/KS)	0.334 ± 0.045	0.633 ± 0.044	0.598 ± 0.023	lọc theo phân vị descriptor
9	UniFault foundation (ngoài)	~ 0.02	~ 0.10	~ 0.03	foundation model 9 tỷ điểm pretrain, fine-tune 10–50 lỗi (§6.3)

Ghi chú std: mỗi ô = recall trung bình $\pm$ độ lệch chuẩn trên 3 seed (42/97/21), cùng $n_{\text{synth}}=3200$. Nguồn: `scripts/15_aggregate_all.py` đọc trực tiếp từ `results/*.json`. Độ lệch chuẩn thấp (hầu hết ≤ 0.05) xác nhận thứ tự xếp hạng **ổn định**, không phải nhiễu seed.

Phương án chính (số 1) đã nâng qua A/B Mục 4 (SPEC §3.4.8): trộn 6 cửa sổ căn pha ($k_{\text{mix}}=6$) nâng $K=10$ $0.689 \rightarrow$ **0.766** (std $0.053 \rightarrow 0.021$) và hơi cao $K=20/50$ so với bản cũ 2 cửa sổ. Số của các phương án khác (2–8) là bản cũ $k_{\text{mix}}=2$ (chỉ phương án chính được tối ưu qua A/B).

Đây là bảng NHẤT QUÁN — mọi nhánh chạy trên cùng một harness (cùng train/test), chỉ khác phần lỗi giả. (TimeGAN là bộ sinh ngoài, kém nhất, không phải phương án chọn.) Dòng 9 (UniFault foundation) là **ngoài harness** (fine-tune model riêng, test IMS cân bằng 8100/8100) — chỉ để so **độ lớn thứ tự**, không bằng nhau tuyệt đối; bạn đọc chủ yếu ở §6.3.

Vì sao interpolate/amp_align/interp_align gần nhau? Cả ba đều là **interpolation** (bám manifold lỗi thật) — `interp_align` thêm căn pha để khử triệt tiêu sóng, `amp_align` thêm chuẩn biên độ. Ở $K=10$ `interp_align+amp` vượt `interpolate` rõ (0.766 vs 0.466).

Đọc thứ tự đúng: `interp_align+amp` (0.748 ở $K=20$) $\approx$ `amp_align` (0.733) $>$ `interpolate` (0.671) $>$ `aligned` (0.633) $\gg$ `heuristic` (0.478) $\approx$ `physics` (0.440) $\approx$ `DDPM` (0.435) $\approx$ `TimeGAN`. Ba nhóm dẫn đầu là biến thể interpolation (bám *manifold lỗi thật*); các nhóm generative/physics/heuristic kém hơn hẳn.

Chi tiết trước/sau khi làm sạch số liệu (cùng mức $n_{\text{synth}}=3200$): một số con số sớm được hiệu chỉnh — `interpolate` $K=50$ $0.800 \rightarrow 0.724$, `amp_align` $K=20$ $0.781 \rightarrow 0.733$, `aligned` $K=50$ $0.787 \rightarrow 0.598$; `interp_align` $K=10$ tăng nhẹ ($0.632 \rightarrow 0.689$, vì trước đó nguồn sinh không nhất quán).

Về trọng số lớp (`class_weight="balanced"`): mẫu lỗi được cân bằng về tổng với lớp bình thường nên model không bị lớp normal áp đảo — điều này **có lợi** cho recall (thử với $K=20$ seed42: `balanced` cho recall-sau=0.749 so với 0.710 nếu không cân bằng). Vì áp dụng giống hệt cho mọi bộ sinh, thứ tự xếp hạng giữa các phương pháp vẫn công bằng.

3. Diễn giải từng phương pháp — "cơ bản nó là gì & kết quả ra sao"

Dưới đây mô tả bản chất từng cách tiếp cận và tại sao nó thắng/thua. Số trong ngoặc là recall **đã chuẩn hóa** ở K=20 (trừ khi ghi chú riêng).

3.1. Baseline trước / sau — mốc tham chiếu KHÔNG augment

- **Cách hoạt động:** RF huấn luyện trên train gồm rất nhiều normal + chỉ K lỗi thật. Với K=20 → model gần như chỉ thấy 20 mẫu lỗi → recall thấp (0.383). Với đầy đủ lỗi (bài IMS đủ, ~2000 lỗi) → recall chạm **0.886 / AUC 0.982** (trần của feature kỹ thuật).
- **Kết luận:** đây là "nút thắt dữ liệu", không phải model yếu — bằng chứng ở §3.5 (kể cả self-supervised cũng không vượt).
- **AE (unsupervised, không cần nhãn lỗi):** AE chỉ học "bình thường" + chấm ngưỡng P99 → IMS **precision 0.984 / recall 0.657 / F1 0.788 / AUC 0.912**. Lưu ý số này là **tái chạy sau khi sửa bug split** (bản trước ghi 0.52/0.03 là sai do leak) — baseline hữu ích cho kịch bản "chưa có nhãn lỗi".

3.2. [1] interp_align + amp + k_mix=6 (PHƯƠNG ÁN CHÍNH)

- **Cơ bản là gì:** *Phase-aware mixup* (K=6). Lấy 6 cửa sổ lỗi THẬT, **căn pha cho khớp nhau** bằng FFT cross-correlation rồi trộn theo trọng số Dirichlet thực sự, kèm jitter nhỏ. Việc căn pha trước khi hiện tượng **triệt tiêu sóng mang** khi trộn hai sóng lệch pha (làm bản trộn "dịu/è" đi); trộn 6 cửa sổ (thay vì 2) phủ dày hơn vùng giữa manifold lỗi. Sau đó chuẩn biên độ về khoảng [q10,q90] của lỗi thật (--amp).
- **Kết quả:** K=10: 0.252 → **0.766**; K=20: 0.383 → **0.748**; K=50: 0.490 → **0.788**. Tốt nhất mọi mức khan hiếm (A/B §3.4.8).
- **Vì sao thắng:** nội suy *trong* manifold lỗi thật — không "học" gì, nên **không sụp khi chỉ có vài chục mẫu**; căn pha giữ năng lượng sóng mang; chuẩn biên độ bù mẫu amplitude cực đoan; và `k_mix=6` giúp sinh bản giữa nhiều mẫu → đặc biệt mạnh khi lỗi RẤT hiếm (K=10). Đây là tinh chỉnh cuối, được chọn qua A/B tổng cộng 3 biến thể (SPEC §3.4.8).

3.3. [2] amp_align

- **Cơ bản:** *interpolate* + *chuẩn biên độ*. Trộn 2 lỗi thật + jitter rồi scale từng bản về std trong [q10,q90] của lỗi thật. Bù đúng điểm yếu của interpolate (bản trộn quá "dịu", thiếu amplitude cực đoan).
- **Kết quả:** K=10: **0.678**; K=20: **0.733**; K=50: **0.773**. **Càng hiếm càng giúp.**
- **Nhận xét:** rất gần interp_align (deliverable chọn interp_align vì căn pha nên *ít cần* bù biên độ hơn).

3.4. [3] interpolate (optimized)

- **Cơ bản:** *trộn/khớp giữa các lỗi thật + jitter*. Chọn ngẫu nhiên 2 cửa sổ lỗi thật, trộn tuyến tính `a·x1 + (1-a)·x2 + jitter`.
- **Kết quả:** K=10: **0.466**; K=20: **0.671**; K=50: **0.724**.
- **Nhận xét:** nền tảng của cả amp_align và interp_align. Thắng heuristic/generative nhờ **bám sát manifold lỗi thật**, không trôi ra ngoài.

3.5. Physics (BPFO/BPFI/BSF/FTF) — và bản recalibrate `physics_align`

- **Cơ bản:** mô hình cơ chế hồng ổ bi. Tính tần số đặc trưng (outer/inner race, ball, cage) theo hình học ổ bi, rồi **bơm xung và đập lặp** vào tín hiệu normal: $x_f(t) = x_h(t) + A \cdot \sum h(t - kT) \cdot w(t) + n(t)$.
- **Kết quả:** bộ sinh CŨ (`src/physics.py` , `depth` + tần số va đập ~600Hz cố định) — K=10: **0.411**, K=20: **0.440**, K=50: **0.597**. Gần bằng DDPM/heuristic, xa interpolate.
- **Vì sao thua (đã chẩn đoán, không phải bug):** đo descriptor cho thấy lỗi physics sinh ra **quá "mềm"** so với lỗi IMS thật — std **0.51×**, env_energy **0.31×**, kurtosis **0.36×**. RF học từ lỗi yếu này không nhận ra lỗi thật mạnh. Nguyên nhân: `depth` /tần số cố định, **không chuẩn theo lỗi thật**.
- **physics_align (recalibrate, mới):** `OptimizedFaultGenerator` (resonance + depth học từ lỗi thật) + **blend bớt nền normal** (giảm kurtosis từ ~11× về ~1.9× khớp thật) + `_amp_rescale` về [q10,q90] std lỗi thật → K=10: **0.693**; K=20: **0.665**; K=50: **0.700**. **Nhảy từ ~0.44 lên ~0.66–0.70**, tiến sát interpolate/amp_align, chỉ thua interp_align. → Kết luận: **physics không yếu về bản chất mà do chưa được chuẩn theo manifold lỗi thật** (`scripts/06 --gen physics_align --blend 0.25`).

3.6. DDPM / Diffusion

- **Cơ bản:** *denoising diffusion*. Học cách khử nhiễu để rồi sinh mẫu mới từ nhiễu trắng — mô hình generative "SOTA" theo research 2026.
- **Kết quả:** K=10: **0.432**; K=20: **0.435**; K=50: **0.604**.
- **Vì sao thua:** loss huấn luyện **phẳng ~1.0** — mạng không học được phân bố từ 20–50 mẫu (mode collapse). Diffusion cần **hiều mẫu để học phân bố**; ở kịch bản khan hiếm thì không dùng được.

3.7. TimeGAN (generative)

- **Cơ bản:** *GAN theo thời gian* — học phân bố chuỗi thời gian, gồm generator + discriminator + một bộ điều phối để giữ độ liên mạch của chuỗi.
- **Kết quả:** K=20: ~0.43; K=50: ~0.58. **Kém** trong các bộ sinh.
- **Vì sao thua:** cần nhiều mẫu để học phân bố → 20–50 mẫu → mode collapse. Đây là bộ sinh **đúng keyword đề** nhưng lại yếu nhất ở kịch bản hiếm.

3.8. Heuristic (×2.5 + nhiễu)

- **Cơ bản:** đơn giản nhất — lấy window *normal*, nhân biên độ 2.5× rồi cộng nhiễu → xem như "lỗi".
- **Kết quả:** K=10: **0.492**; K=20: **0.478**; K=50: **0.628**. Chỉ gần baseline+, không mạnh.
- **Vì sao thua:** đây là lỗi **bịa**: chỉ đổi *biên độ* chứ không tạo *cấu trúc lỗi* (điểm khác biệt trước/sau khi hồng là ở dạng sóng/tần số, không chỉ to hơn) → model học lệch..

3.9. Các nhánh đã thử nhưng BỎ (không giúp / có hại)

Phương pháp	Cơ bản là gì	Kết quả	Quyết định
ECDF/KS distribution alignment	Đóng "sim-to-real gap": lọc bản sinh theo dải phân vị của 7 descriptor lỗi thật	K=20: 0.633, K=50: 0.598 — thấp hơn interpolate (0.671/0.724)	Không thay được interpolate → bỏ
TTA (test-time augmentation)	Mỗi window test tạo 8 view (nhiều/lệch pha/scale) rồi vote	recall GIẢM (K=50 -0.022, K=20 -0.013)	RF không invariance → bỏ
Ensemble đa seed RF	Vote 5 RF seed khác nhau	$\Delta \approx 0$	RF 200 cây đã ổn định → bỏ

Phương pháp	Cơ bản là gì	Kết quả	Quyết định
Self-supervised contrastive	Học biểu diễn không nhãn (SimCLR 1D-CNN) rồi RF trên embedding	$0.358 < 0.418$ (feature 24D)	feature kỹ thuật đã đủ mạnh → bỏ
SMOTE trong feature space	Nội suy trong space 24D	0.637	mất cấu trúc thời gian → kém

3.10. Feature engineering "vô địch Kaggle" (bỏ)

- **Peak-over-noise-floor (pnf)** + 4 feature: 0.785 vs 0.786 (trong nhiễu) — **bỏ**.
- **VSB 1st thật** (flatiron + knee-point + sawtooth + 7 peak features): `sawtooth_rmse` sep=0.06 (≈ 0) → **không chuyển được sang window 512/42ms**; `peak_height` tách mạnh nhưng **trùng thông tin** feature đã có → +0.004 trong nhiễu → **bỏ**.
- **Kết luận:** feature kỹ thuật 24D đã đạt trần (AUC 0.99); phần dư địa nằm ở **số lượng mẫu lỗi**, không phải feature — bài ố bi khác bài partial-discharge của VSB.

4. Bảng chuẩn hóa — kết quả theo từng mức khan hiếm (mean 3 seed)

Số lỗi thật K	Baseline	interp_align+amp	amp_align	interpolate	heuristic	physics	DDPM	TimeGAN
10	0.252	0.766 ± 0.021	0.678 ± 0.058	0.466 ± 0.009	0.492 ± 0.031	0.411 ± 0.031	0.432 ± 0.035	—
20	0.383	0.748 ± 0.012	0.733 ± 0.010	0.671 ± 0.061	0.478 ± 0.029	0.440 ± 0.007	0.435 ± 0.009	~0.43
50	0.490	0.788 ± 0.025	0.773 ± 0.031	0.724 ± 0.038	0.628 ± 0.018	0.597 ± 0.009	0.604 ± 0.009	~0.58

Toàn bộ là số đã kiểm chứng nhất quán (không trộn các bộ số khác nhau). Nguồn: `results/fix_*` (interp_align, physics) và `results/fx_*` (các nhánh còn lại), mỗi ô = mean 3 seed (42/97/21).

Kết luận: ở mọi mức khan hiếm, interp_align+amp dẫn đầu; interpolation (các nhánh 1–3) vượt xa generative/physics/heuristic. Trong kịch bản thiếu lỗi thật, cách tốt nhất là **bám manifold lỗi thật (interpolation)**.

5. Phát hiện quan trọng trong quá trình research (giá trị cho slide)

1. **Lỗi dữ liệu, không phải lỗi model (quan trọng nhất).** Trước khi sửa split, mọi model kẹt ở AUC ~0.58. Nguyên nhân: `limit` cắt 300 file ĐẦU rồi mới chia vùng 90–100% — nhưng quá trình

hỏng nằm ở **CUỐI test** (file ~2000) → nhãn "fault" thực chất là normal chưa hỏng. **Sửa: chia vùng theo toàn bộ chuỗi**. Kết quả: AUC 0.582 → **0.982**.

2. **Augmentation chỉ có giá trị khi THẬT SỰ hiếm lỗi**. Với $\geq 100-200$ lỗi thật, baseline chạm trần, augment ≈ 0 . Với 10–50 lỗi, augment rất đáng giá.
3. **Nhiều lỗi giả không phải lúc nào cũng tốt (n_{synth} không đơn điệu)**. $K=10 + n_{\text{synth}} 8000 \rightarrow$ recall **tụt -0.143** ($0.632 \rightarrow 0.489$) do các bản trộn trùng manifold. → **Giữ $n_{\text{synth}} \approx 3200$ là tối ưu**.
4. **Vì sao nên ưu tiên "interpolate lỗi thật" hơn "mô hình sinh phân bố"**. Khi chỉ có vài chục mẫu, "học phân bố" (GAN/diffusion) suy sụp; "nội suy trong manifold lỗi thật" thì không — vì không cần học.

6. Hai kiểm chứng bổ sung — đối chứng mô hình tự-học & đánh giá trên lỗi sớm

Đây là hai câu hỏi then chốt mà objective đặt ra, nay đã trả lời bằng số liệu trên **cùng hàng rào đánh giá khan hiếm**.

6.1. Self-supervised / contrastive — ngang bằng baseline feature, KÉM XA deliverable

Chạy lại `ContrastiveEncoder` (SimCLR, 1D-CNN, 6000 windows không nhãn, 25 epochs, NT-Xent hội tụ ~1.2–4.4) đúng trên cùng train/test khan hiếm; RF trên embedding 128D so với feature 24D. **Chú ý: số dưới là kết quả sau khi sửa bug (dùng keep-toàn-bộ-fault thay vì subsample ngẫu nhiên 6000 từ ~40k cửa sổ — subsample cũ có thể làm rơi hết K lỗi thật khi K nhỏ)**.

Số lỗi thật K	Baseline (24D)	Contrastive (embedding)
10	0.173 $\pm$ 0.068	0.196 $\pm$ 0.151
20	0.374 $\pm$ 0.024	0.289 $\pm$ 0.042
50	0.512 $\pm$ 0.023	0.481 $\pm$ 0.046

Đối chứng (sau khi sửa bug): contrastive **ngang bằng** feature 24D ở $K=10$ (0.196 vs 0.173), và **hơi thua** ở $K=20/50$ (0.289/0.481 vs 0.374/0.512) — tức **tự-học biểu diễn không tạo ra lợi thế so với feature kỹ thuật**, đồng thời vẫn **rất xa** deliverable interp_align+amp (0.766/0.748/0.788). Kết luận đối chứng objective được giữ, nhưng **không còn thái quá**: contrastive không "kém xa" mà chỉ **không hơn** baseline feature; đường hiệu quả vẫn là sinh lỗi giả bám manifold lỗi thật.

⚠ Bản cũ §6.1 (0.000/0.071/0.057) là SAI do bug subsample — bị loại. Số đúng ở bảng trên.

6.2. Lỗi sớm (incipient fault) — hàng rào khó nhưng augmentation VẪN giúp

Harness gốc chỉ đánh giá **lỗi hỏng nặng cuối vòng đời** (file cuối). Predictive-maintenance thực tế quan tâm **phát hiện sớm** — nên tôi dựng thêm kịch bản lỗi **mới chớm/phân hủy nhẹ** (vùng 90–95% chuỗi, biên độ thấp) làm fault-test, huấn luyện y hệt như trên:

Số lỗi thật K	Baseline	interp_align+amp	Độ tăng
10	0.004	0.116 ±0.015	+0.112
20	0.012	0.097 ±0.010	+0.085
50	0.034	0.127 ±0.024	+0.093

Ba điều quan trọng từ kết quả này (số dùng k_mix=6 — deliverable §3.4.8): 1. **Lỗi sớm RẤT khó** — cả baseline lẫn augment đều có recall thấp (0.004–0.13). Đây là giới hạn thực của dữ liệu (vùng 90–95% tín hiệu chưa đủ mạnh để phân biệt rõ với normal suy giảm). 2. **Dù vậy, augmentation VẪN tăng recall ~20–30×** (0.004 → 0.12), tức vẫn đóng góp giá trị rõ ở dải phổ biến khó — không phải "chỉ giúp khi lỗi dễ". 3. **Câu chuyện "khan hiếm lỗi" được khẳng định kép:** đối với lỗi nặng, augment nâng recall đến ~0.79; đối với lỗi sớm, augment nâng recall tuyệt đối +0.09–0.11 (~20–30× baseline).

Kết luận tổng hợp để trả lời objective: đối chứng foundation/contrastive (6.1) cho thấy **not-winning** — feature kỹ thuật 24D + interpolation vẫn là cách tốt nhất ở kịch bản hiếm. Kịch bản lỗi sớm (6.2) cho thấy **augmentation có giá trị kể cả ở dải khó**, dù giới hạn cứu được là có thật (lỗi sớm biên độ quá nhỏ thì mô hình nào cũng bó tay).

6.3. Foundation model (UniFault) — cũng THẤT BẠI ở kịch bản hiếm

Đối chứng mạnh nhất: dùng **UniFault** — foundation model chẩn đoán lỗi ổ bi đã pretrain trên 9 tỷ điểm tín hiệu (github.com/emadeldeen24/UniFault) — rồi fine-tune từng mức khan hiếm **đúng cùng kịch bản K=10/20/50** trên IMS. Đây là cột "SOTA foundation" để xem pretrain có cứu được few-shot hay không (kỳ vọng marketing là CÓ).

- **Thiết lập:** IMS → cửa sổ 1024 (stride 512, kênh DE), fine-tune pretrained **tiny** (819K params, matched 80/83 trọng số) và **base** (30M, train-from-scratch) 30 epochs/patience 20 trên train 36k mẫu (chỉ K lỗi), test cân bằng 8100 normal/8100 fault. Harness riêng (không phải RF+feature của các phương pháp trên).

Số lỗi thật K	UniFault tiny (foundation)	UniFault base (from scratch)
10	0.020 ±seed	0.000
20	0.102	0.000
50	0.000–0.047 (biến thiên seed)	0.000

Kết luận rất quan trọng: foundation model **cũng không cứu được few-shot**. Dù pretrain trên 9 tỷ điểm, fine-tune chỉ với 10–50 lỗi → recall fault chỉ **0.02–0.10** (gần như chỉ đoán normal). Base 30M từ-scratch còn tệ hơn (0.000 — overfit 2-lớp). Kết quả này **khẳng định mạnh luận điểm trung tâm của báo cáo:** ở kịch bản khan hiếm, **không phải model mạnh hay pretrain giải quyết vấn đề** — mà là **bổ sung lỗi giả** (interpolation bám manifold lỗi thật đạt 0.766/0.748/0.788, vượt foundation ~7–38×). Và nó cho thấy "nhiều lỗi giả" (của mình) đánh bại "pretrain khổng lồ" (của UniFault).

⚠ **Lưu ý độ tin cậy:** UniFault dùng test cân bằng 8100/8100 (khác test-RF của các nhánh 1-8), nên **chỉ so được về độ lớn thứ tự, không bằng nhau tuyệt đối**. Giá trị là ở hướng: kể cả foundation SOTA cũng kẹt ở ~0.1 recall khi chỉ có <=50 lỗi.

6.4. Generative (DDPM) + quality-gating cho lỗi sớm — vẫn THUA interpolation

Sau khi DDPM thua mọi phương pháp ở lỗi nặng (§2), tôi thử một lần cuối cho kịch bản **lỗi sớm** (điểm yếu duy nhất của deliverable, recall chỉ 0.10–0.13): train `LightDDPM` (ddpm.py, GPU) trên K lỗi thật rồi sinh lỗi giả, kèm **quality-gating** — lọc các bản sinh bằng 7 descriptor (std/rms/kurt/spec_centroid/env_energy/A@f_char/sideband_snr), chỉ giữ bản nằm trong dải [q10,q90] của lỗi thật để loại bản méo/trống do mode-collapse.

Số lỗi thật K	Baseline	DDPM thô	DDPM + quality-gate	interp_align+amp
10	0.006	0.008	0.009	0.127
20	0.023	0.015	0.019	0.103
50	0.030	0.032	0.033	0.141

Kết luận: generative **vẫn thua** ở lỗi sớm — DDPM recall ngang baseline (0.01–0.03), **còn tệ hơn interp_align 4–14×**. Nguyên nhân gốc: **DDPM mode-collapse** — loss huấn luyện bị kẹt ~1.0 (không giảm dù tăng epochs 60 → 150) vì chỉ có 10–50 mẫu (đúng cảnh báo ddpm.py §3.4).

Quality-gating gần như không giúp (0.009 vs 0.008; 0.019 vs 0.015; 0.033 vs 0.032) — vì các bản DDPM sinh ra đều trải một dải descriptor rất hẹp quanh normal, lọc không còn bản "lỗi" để giữ.

Giá trị của thí nghiệm này (rất đáng cho slide): nó **đóng ngoặc** mọi hướng generative — GAN (TimeGAN, kém), diffusion (DDPM, kém), foundation pretrain (UniFault, kém), kể cả khi thêm bộ lọc chất lượng (gating, không giúp). Ở kịch bản **cực khan hiếm lỗi**, đường thẳng duy nhất được xác nhận là **nội suy trong manifold lỗi thật** (interpolate family). Đây là luận điểm trung tâm, nay được củng cố bằng đủ mọi phía (model mạnh, generative, foundation).

7. Code / tác giả chịu trách nhiệm (để đối chiếu)

Phương pháp	File bộ sinh	Script đánh giá
interp_align+amp	src/generator.py::generate_aligned_interp + _amp_rescale	scripts/06_scarcity_aug.py --gen interp_align --amp 1
amp_align	generate_amp_aligned	--gen amp_align
interpolate	generate_interpolated	--gen optimized
physics	src/physics.py + generator	--gen physics
DDPM	src/ddpm.py	--gen ddpm
DDPM + quality-gating (lỗi sớm)	scripts/16_ddpm_gatingincipient_ab.py	results/incip_ddpm_ab.json

Phương pháp	File bộ sinh	Script đánh giá
TimeGAN	src/timegan_torch.py	--gen npy
TTA/ensemble	—	scripts/09_tta_ensemble.py
contrastive	src/contrastive.py	scripts/08_contrastive_vs_aug.py , scripts/13_contrastive_harness.py
UniFault foundation	unifault_local/ (UniFault clone + build_parquet.py , run_unifault.py)	unifault_local/results.txt
lỗi sớm (incipient)	—	scripts/14_incipient_ab.py
baseline split	src/pipeline.py	—

8. Kết luận & khuyến nghị cho bản nộp Vòng 1

Phương án chính thức:

```
--gen interp_align --amp 1 --n-synth 3200 --k-mix 6
```

= **interpolate lỗi thật** (lỗi) + **căn pha cross-correlation** (khử triệt tiêu sóng mang) + **chuẩn biên độ** (bù amplitude cực đoan) + **trộn 6 cửa sổ** (phủ dày manifold tối ưu ở mức hiếm) + **đủ số lượng lỗi giả (3200)**.

Bảng tóm tắt recall (mean 3 seed):

Số lỗi thật	Baseline	Sau augment	Độ tăng
10	0.252	0.766	+0.516
20	0.383	0.748	+0.364
50	0.490	0.788	+0.266

Quy trình nghiên cứu (tóm tắt): Baseline RF+feature → nhận ra loss vì dữ liệu (fix split) → thử heuristic/physics/TimeGAN (tất cả kém) → phát hiện "khan hiếm lỗi thật" mới là bản chất đề → thử interpolate (thắng) → amp_align → interp_align (căn pha, thắng nhất) → research thêm feature (VSB/denoise, không giúp) → thử diffusion/contrastive (không hơn) → đối chứng cũng-harness (contrastive ≈ baseline, kém xa deliverable; **UniFault foundation SOTA cũng thất bại ~0.02-0.10**) + kịch bản lỗi sớm (augment vẫn giúp 20-25×) → A/B tinh chỉnh interp_align (chốt k_mix=6) → **chốt interp_align+amp+k_mix=6** với số chuẩn 0.77/0.75/0.79.

Tổng kết trả lời objective (đã khép kín): - So với contrastive/foundation ở cùng mức khan hiếm: interp_align+amp **thắng rõ** (contrastive ≈ baseline feature, kém xa deliverable — §6.1; **UniFault foundation SOTA chỉ đạt ~0.02-0.10, thua ~7-38× — §6.3**). Đây là cột so sánh then chốt của đề, nay đã có số. - So với lỗi sớm (incipient): cả baseline lẫn augment đều khó (giới hạn thật của dữ liệu), nhưng augment vẫn nâng recall **~20-30×** trong tất cả mức K — chứng tỏ giá trị của nó không chỉ ở lỗi "dễ".